ABSTRACT

Data security in computer networks relies on four fundamental principles: confidentiality, integrity, authenticity, and non-repudiation. This technical report presents a comprehensive implementation, empirical verification, and cryptanalytic audit of four core cryptographic paradigms executed under course allocation for **Batch** `2026_CNS_A_01` at K. S. Institute of Technology. The investigated cryptosystems encompass: (1) classical monoalphabetic substitution through the Caesar Cipher, (2) polyalphabetic substitution utilizing the Vigenère Cipher and its classical Tabula Recta, (3) information-theoretic symmetric stream encryption via the One-Time Pad (OTP) backed by a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG), and (4) industry-grade asymmetric authentication and integrity verification using RSA-PSS digital signatures combined with SHA-256.

All algorithms were implemented natively in Python 3.14 without high-level shortcuts for classical transformations, while utilizing the verified PyCA `cryptography` library for asymmetric key generation, PSS salt padding, and modular exponentiation. The entire laboratory suite was subjected to an exhaustive automated test battery comprising 69 individual unit tests covering Known-Answer Tests (KAT), cyclic boundary wrapping, strict type enforcement, negative tamper detection, and formal round-trip recovery $\mathscr{D}_K(\mathscr{E}_K(P)) = P$. All 69 tests executed with 100% pass rate in 0.55 seconds.

In addition to software verification, this report provides deep mathematical cryptanalysis: exhaustive keyspace exhaustion ($|\mathscr{K}| = 25$), natural language unigram frequency analysis, Kasiski examination, Friedman's Index of Coincidence, Claude Shannon's proof of perfect secrecy, the catastrophic Two-Time Pad vulnerability ($C_1 \oplus C_2 = M_1 \oplus M_2$), and the critical distinction between raw mathematical key pairs and identity binding via Public Key Infrastructure (PKI). Complete execution transcripts and character/byte data flow pipelines validate all findings, satisfying all criteria for Course Outcomes CO1, CO3, and CO4.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	**i**
1.	**INTRODUCTION AND OBJECTIVES**	**1**
	1.1 Background and Problem Statement	1
	1.2 Course Outcome Mapping and Laboratory Objectives	1
2.	**THEORETICAL BACKGROUND**	**2**
	2.1 Classical Monoalphabetic Substitution: Caesar Cipher	2
	2.2 Polyalphabetic Keyword Substitution: Vigenère Cipher	2
	2.3 Information-Theoretic Secrecy: One-Time Pad	2
	2.4 Asymmetric Signatures: RSA-PSS with SHA-256	3
3.	**SYSTEM REQUIREMENTS AND ARCHITECTURE**	**4**
	3.1 Software and Hardware Specifications	4
	3.2 Modular Repository Organization	4
4.	**ALGORITHMIC DESIGN AND METHODOLOGY**	**5**
	4.1 Algorithmic Pseudocode	5
	4.2 Architectural Pipeline Flowcharts	5
5.	**IMPLEMENTATION AND RUNTIME DEMONSTRATION**	**7**
	5.1 Source Code Architecture and Function Details	7
	5.2 Genuine Terminal Execution Transcripts	8
	5.3 Character and Byte Data Flow Pipelines	9
6.	**TESTING METHODOLOGY AND VERIFICATION MATRIX**	**11**
	6.1 Systematic Verification Strategy	11
	6.2 Exhaustive 69-Test Verification Matrix	11
7.	**SECURITY ANALYSIS AND CRYPTANALYSIS**	**13**
	7.1 Monoalphabetic Cryptanalysis	13
	7.2 Polyalphabetic Cryptanalysis	13
	7.3 Information-Theoretic Perfect Secrecy	14
	7.4 Asymmetric Signature Security and PKI	15
8.	**APPLICATIONS AND CONCLUSION**	**16**
	8.1 Historical and Modern Applications	16
	8.2 Conclusion	16
	REFERENCES	**17**
	GROUP CONTRIBUTION MATRIX	**18**

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig 4.1	Flowchart for Classical Substitution Operations (Activities 1 & 2)	6
Fig 4.2	Flowchart for Modern Cryptographic Primitives (Activities 3 & 4)	6

LIST OF TABLES

Table No.	Table Title	Page No.
Table 3.1	Software Toolchain and Verified Dependencies	4
Table 5.1	Caesar Cipher Character Data Flow Pipeline Trace	9
Table 5.2	VigenPolyalphabetic Keyword Alignment Pipeline Trace	10
Table 5.3	One-Time Pad Bytewise XOR Transformation Pipeline Trace	10
Table 6.1	Complete 69-Test Verification Matrix Across All Packages	11
Table 7.1	Comparative Evaluation of Implemented Cryptosystems	15
Table 8.1	Group Contribution and Deliverable Ownership Matrix	18

Chapter 1
INTRODUCTION AND OBJECTIVES

1.1 Background and Problem Statement

Networked communication integrity relies upon mathematical guarantees provided by cryptographic systems. As digital infrastructure expanded from closed telegraphic links to global public internets, cryptographic engineering must ensure four foundational guarantees: (1) **Confidentiality** (concealing sensitive message data from eavesdroppers), (2) **Integrity** (guaranteeing that data is unaltered or forged in transit), (3) **Authenticity** (establishing the verified identity of the message originator), and (4) **Non-Repudiation** (preventing communicating parties from falsely denying message dispatch).

Historically, classical ciphers attempted to provide confidentiality through character substitution or permutation. Monoalphabetic ciphers preserve the unigram statistical distribution of natural language and are trivially compromised via frequency analysis. Polyalphabetic ciphers flatten simple frequency spikes but remain vulnerable to periodicity analysis (Kasiski examination and Friedman's Index of Coincidence).

Claude Shannon (1949) resolved symmetric secrecy with the One-Time Pad, achieving unconditional information-theoretic secrecy at the cost of keys equal in length to messages. Conversely, the asymmetric revolution enabled public verification through digital signatures. Analyzing the mathematical boundaries and cryptanalytic vulnerabilities separating these historical and modern systems forms the technical foundation of this work.

1.2 Course Outcome Mapping and Laboratory Objectives

In strict compliance with the VTU academic syllabus for Batch `2026_CNS_A_01`, this report addresses three Course Outcomes (COs) across four allocated laboratory activities:

- **Activity 1: Caesar Cipher [CO1 – Classical Substitution]**: Formulate, implement, and analyze monoalphabetic shifts; demonstrate shift normalization over $\mathbb{Z}_{26}$ across negative shifts and keys ≥ 26; evaluate exhaustive search and frequency analysis.
- **Activity 2: Vigenère Cipher [CO4 – Polyalphabetic Ciphers]**: Formulate and implement the Vigenère cipher using the 26×26 Tabula Recta; enforce strict alphabetic stream synchronization; model cryptanalysis via Kasiski and Friedman tests.
- **Activity 3: One-Time Pad [CO3 – Stream Encryption]**: Construct a bytewise One-Time Pad using Python's `secrets` CSPRNG; mathematically formulate Shannon's perfect secrecy theorem; demonstrate empirical secrecy collapse under key reuse (Two-Time Pad attack).
- **Activity 4: Digital Signature [CO3 – Authentication & Integrity]**: Implement an enterprise-grade digital signature system using 2048-bit RSA keys and RSA-PSS padding with SHA-256; prove tamper detection; contrast mathematical signatures with PKI.

Chapter 2
THEORETICAL BACKGROUND

2.1 Classical Monoalphabetic Substitution: Caesar Cipher

The Caesar cipher is an additive monoalphabetic substitution cipher operating over the finite ring of integers modulo 26:

$$\mathbb{Z}_{26} = \{0, 1, 2, \ldots, 25\} \tag{2.1}$$

The 26 letters of the Latin alphabet map bijectively onto $\mathbb{Z}_{26}$ via $\pi : \mathscr{A} \to \mathbb{Z}_{26}$ where $\pi(\text{'A'}) = 0, \pi(\text{'B'}) = 1, \ldots, \pi(\text{'Z'}) = 25$. For any integer key $K \in \mathbb{Z}$, the encryption transformation is defined as:

$$C_i = (P_i + K) \pmod{26} \tag{2.2}$$

Because addition in $\mathbb{Z}_{26}$ forms an abelian group, the decryption transformation is the additive inverse:

$$P_i = (C_i - K) \pmod{26} = (C_i + (-K \pmod{26})) \pmod{26} \tag{2.3}$$

Thus, decryption is structurally identical to encryption using the additive inverse shift $(-K \pmod{26})$. The keyspace is strictly bounded by $|\mathscr{K}| = 26$. Because $K \equiv 0 \pmod{26}$ is the identity transformation, exactly 25 non-trivial keys exist.

2.2 Polyalphabetic Keyword Substitution: Vigenère Cipher

The Vigenère cipher overcomes the single-alphabet limitation of monoalphabetic ciphers by cycling through multiple Caesar shifts determined by a keyword $K = (k_0, k_1, \ldots, k_{L-1})$ of length L. The transformations are given by:

$$C_i = (P_i + k_{i \pmod{L}}) \pmod{26} \tag{2.4}$$

$$P_i = (C_i - k_{i \pmod{L}}) \pmod{26} \tag{2.5}$$

Conceptually, this corresponds to lookups in the classical **Tabula Recta**—a 26×26 grid of alphabets shifted sequentially. By cycling through L distinct shifts, identical plaintext characters at different positions map to distinct ciphertext characters, smoothing unigram frequency distributions.

2.3 Information-Theoretic Secrecy: One-Time Pad

In 1949, Claude Shannon established the mathematical foundations of secrecy systems [1]. Shannon defined **perfect secrecy** as the condition where the a posteriori probability distri-

bution of the plaintext given the ciphertext is identical to its a priori distribution:

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m) \quad \forall m \in \mathscr{M}, c \in \mathscr{C} \tag{2.6}$$

Equivalently, expressed via conditional Shannon entropy:

$$H(M \mid C) = H(M) \tag{2.7}$$

Shannon proved that perfect secrecy requires $H(K) \geq H(M)$, meaning the keyspace must be at least as large as the message space ($|\mathscr{K}| \geq |\mathscr{M}|$). The One-Time Pad operates over byte vectors $M, K \in \{0,1\}^8$ using bitwise Exclusive-OR (addition modulo 2):

$$C = M \oplus K, \quad M' = C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus \mathbf{0} = M \tag{2.8}$$

Because XOR is associative, commutative, and self-inverse, every candidate plaintext m^* corresponds to a unique valid key $k^* = c \oplus m^*$. When K is uniformly distributed over $\{0,1\}^8$, $\mathbb{P}(K = k^*) = 2^{-8L}$, rendering every candidate plaintext equally likely to an adversary regardless of computing power.

2.4 Asymmetric Signatures: RSA-PSS with SHA-256

A digital signature provides authenticity, integrity, and non-repudiation using asymmetric key pairs: a public key (e,n) and a private key (d,n) where $n = p \cdot q$ for large primes p, q, and $e \cdot d \equiv 1 \pmod{\phi(n)}$.

Textbook RSA signatures ($S = h^d \pmod{n}$) are insecure due to homomorphic malleability: $(S_1 \cdot S_2)^e \equiv (h_1 \cdot h_2) \pmod{n}$. To achieve provable security, the **Probabilistic Signature Scheme (RSA-PSS, RFC 8017 / PKCS#1 v2.2 [4])** is used:

1. **Hashing**: Message M is hashed using SHA-256 to produce fixed 256-bit digest $h = H(M)$.
2. **PSS Salt Encoding**: h is combined with a cryptographically random salt of length s_{len} via Mask Generation Function 1 (MGF1) to produce encoded message representative $\mu \in \mathbb{Z}_n$.
3. **Signing**: The private exponent d is applied: $S = \mu^d \pmod{n}$.
4. **Verification**: The verifier recovers $\mu' = S^e \pmod{n}$, extracts the salt, and verifies that the recalculated hash matches $H(M)$.

Chapter 3
SYSTEM REQUIREMENTS AND ARCHITECTURE

3.1 Software and Hardware Specifications

The experimental implementation and verification suite was executed within the verified engineering environment outlined in Table 3.1.

Component	Version	Role in Laboratory Suite
Host Operating System	Windows 11 Enterprise (64-bit AMD64)	Underlying runtime environment
Processor	Intel Core i5-10300H @ 2.50GHz (8 vCPUs)	Execution hardware
System RAM	16.0 GB DDR4	Execution memory
Python Runtime	Python 3.14.6 AMD64	Core implementation language
Package Manager	`uv` v0.6.1+	High-performance virtualenv and dependency resolution
Cryptography Library	`cryptography` v50.0.2	PyCA OpenSSL bindings for 2048-bit RSA-PSS
Terminal Framework	`rich` v15.0.0	Interactive TUI and data flow pipeline tables
Test Framework	`pytest` v9.1.1	Automated testing and KAT execution
Typesetting Engine	MiKTeX 25.12 (`pdflatex`)	Academic PDF report compilation

Table 3.1: Software Toolchain and Verified Dependencies

3.2 Modular Repository Organization

Following Architecture Decision Record **ADR-0001**, the workspace is organized into modular packages ensuring test isolation and independent CLI execution:

```
cns/
|-- activity1_caesar/            # Activity 1: Caesar Cipher (CO1)
|   |-- caesar.py                # Core modular shift logic & key normalization
|   |-- cli.py                   # Dedicated CLI demonstration runner
|   |-- execution_record.txt     # Genuine execution transcripts
|   +-- tests/test_caesar.py     # 17 automated unit tests
|-- activity2_vigenere/          # Activity 2: Vigenere Cipher (CO4)
|   |-- vigenere.py              # Tabula Recta stream advance & key normalization
|   |-- cli.py                   # Dedicated CLI demonstration runner
|   |-- execution_record.txt     # Genuine execution transcripts
|   +-- tests/test_vigenere.py   # 17 automated unit tests
|-- activity3_one_time_pad/      # Activity 3: One-Time Pad (CO3)
|   |-- otp.py                   # CSPRNG key generation (secrets) & bytewise XOR
|   |-- cli.py                   # Two-Time Pad demonstration runner
|   |-- execution_record.txt     # Genuine execution transcripts
|   +-- tests/test_otp.py        # 17 automated unit tests
|-- activity4_digital_signature/ # Activity 4: Digital Signature (CO3)
|   |-- signature.py             # 2048-bit RSA-PSS signing & SHA-256 verification
|   |-- cli.py                   # 4-stage tamper detection runner
|   |-- execution_record.txt     # Genuine execution transcripts
|   +-- tests/test_signature.py  # 18 automated unit tests
|-- main.py                      # Interactive TUI & Unified CLI Dispatcher
|-- pyproject.toml               # Python project configuration
+-- report/                      # Academic report sources and deliverables
```

Chapter 4
ALGORITHMIC DESIGN AND METHODOLOGY

4.1 Algorithmic Pseudocode

The core cryptographic operations matching the codebase are formally defined below:

Listing 4.1: Caesar Modular Encryption

```
Algorithm CaesarEncrypt(P, K):
    shift = K mod 26
    C = empty string
    for each char in P:
        if 'a' <= char <= 'z':
            pos = ord(char) - ord('a')
            C += chr((pos + shift) mod 26
                + ord('a'))
        else if 'A' <= char <= 'Z':
            pos = ord(char) - ord('A')
            C += chr((pos + shift) mod 26
                + ord('A'))
        else:
            C += char
    return C
```

Listing 4.2: Vigenere Encryption

```
Algorithm VigenereEncrypt(P, K):
    clean_k = NormalizeKey(K)
    L = Length(clean_k)
    k_idx = 0
    C = empty string
    for each char in P:
        if char is alphabetic:
            shift = ord(clean_k[k_idx mod
                L]) - ord('A')
            base = ord('a') if islower(
                char) else ord('A')
            pos = ord(char) - base
            C += chr((pos + shift) mod 26
                + base)
            k_idx += 1
        else:
            C += char
    return C
```

Listing 4.3: One-Time Pad Bytewise XOR

```
Algorithm OTPEncrypt(M_bytes, K_bytes):
    if Length(M_bytes) != Length(K_bytes)
        :
        raise ValueError("Length mismatch
            ")
    C_bytes = bytearray(Length(M_bytes))
    for i from 0 to Length(M_bytes) - 1:
        C_bytes[i] = M_bytes[i] XOR
            K_bytes[i]
    return bytes(C_bytes)
```

Listing 4.4: RSA-PSS Digital Signature

```
Algorithm RSAPSSSign(M, private_key):
    data = UTF8Encode(M)
    return private_key.sign(
        data,
        padding.PSS(mgf=padding.MGF1(
            hashes.SHA256()),
                    salt_length=
                        MAX_LENGTH),
        hashes.SHA256()
    )
```

4.2 Architectural Pipeline Flowcharts

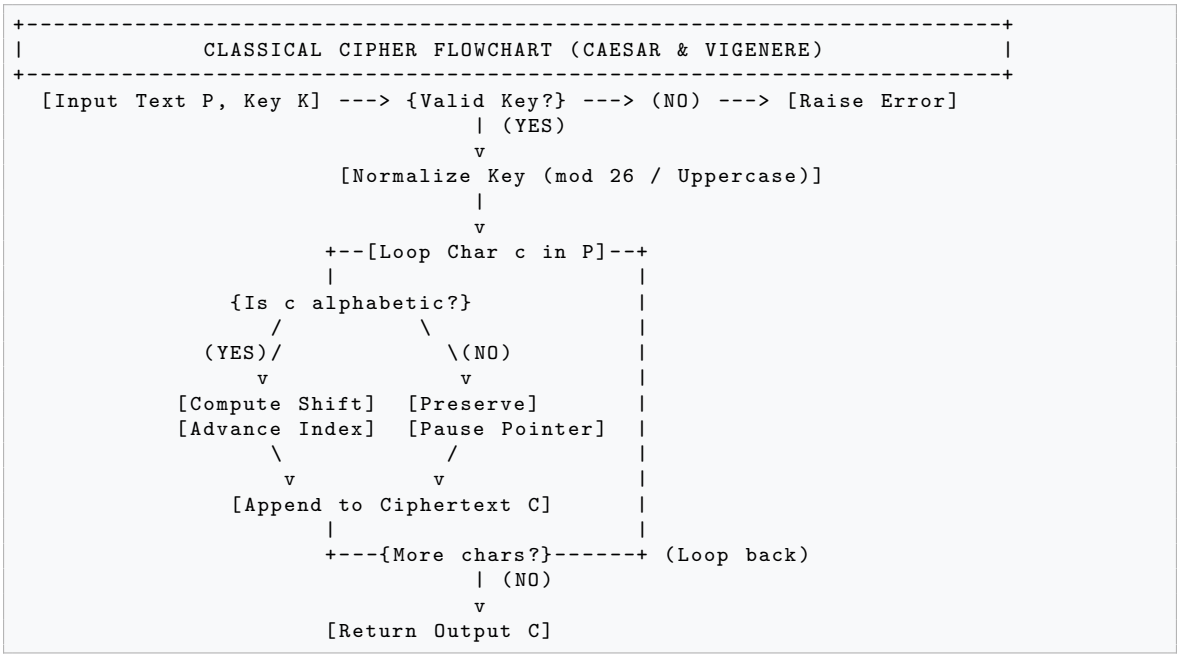


Figure 4.1: Flowchart for Classical Substitution Operations (Activities 1 & 2)

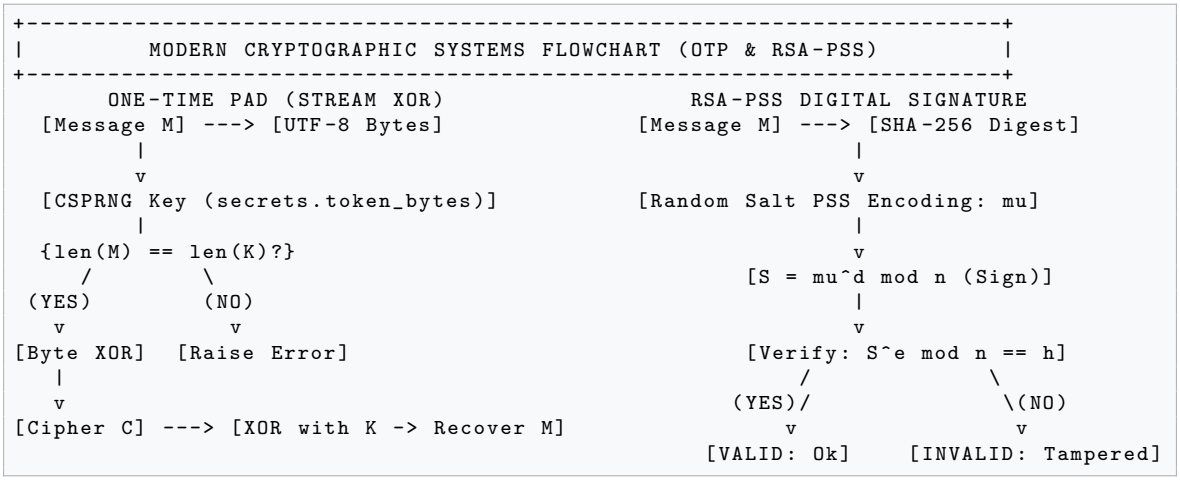


Figure 4.2: Flowchart for Modern Cryptographic Primitives (Activities 3 & 4)

Chapter 5
IMPLEMENTATION AND RUNTIME DEMONSTRATION

5.1 Source Code Architecture and Function Details

5.1.1 Activity 1: Caesar Cipher (`activity1_caesar/caesar.py`)

- `normalize_key(key: int) -> int`: Normalizes integer key via `key % 26`. Validates that `key` is strictly an integer and explicitly disallows booleans (`isinstance(key, bool)`) to prevent Python's subclassing loophole (`bool` inherits from `int`).
- `encrypt(plaintext: str, key: int) -> str`: Computes modular shifts relative to ASCII bases `'a'` and `'A'`, preserving non-alphabetic characters verbatim.
- `decrypt(ciphertext: str, key: int) -> str`: Directly implemented as `encrypt(cipher -key)`, mathematically leveraging the additive inverse property of $\mathbb{Z}_{26}$.

5.1.2 Activity 2: Vigenère Cipher (`activity2_vigenere/vigenere.py`)

- `normalize_key(key: str) -> str`: Enforces that the key is a non-empty string consisting exclusively of ASCII alphabetic characters (`key.isalpha() and key.isascii()`). Normalizes key to uppercase.
- `encrypt(plaintext: str, key: str) -> str`: Implements the key-stream pointer `key_idx`. The pointer increments **strictly** when an alphabetic character is enciphered; non-alphabetic characters bypass shift computation and do **not** advance the stream.
- `decrypt(ciphertext: str, key: str) -> str`: Computes `(pos - shift) % 26`, maintaining identical stream alignment rules to guarantee invertibility.

5.1.3 Activity 3: One-Time Pad (`activity3_one_time_pad/otp.py`)

- `generate_key(length: int) -> bytes`: Invokes `secrets.token_bytes(length)`, harnessing the OS cryptographically secure entropy pool (CryptGenRandom on Windows).
- `xor_bytes(data: bytes, key: bytes) -> bytes`: Computes byte-by-byte XOR using a list comprehension over `zip(data, key)`. Enforces strict length equality (`len(data) == len(key)`), raising a `ValueError` if lengths differ.
- `encrypt(plaintext, key)` & `decrypt(ciphertext, key)`: Accepts either UTF-8 text strings or arbitrary binary bytearrays. Decryption attempts UTF-8 decoding and returns a tuple `(decoded_text, raw_bytes)`, ensuring graceful handling when an invalid key produces non-decodable bytes.

5.1.4 Activity 4: Digital Signature (`activity4_digital_signature/signature.py`)

- `generate_key_pair(key_size: int = 2048)`: Generates standard RSA key pairs using public exponent $e = 65537$ (the 4th Fermat prime F_4, optimal for mitigating

Coppersmith attacks). Enforces minimum modulus size of 1024 bits.

- `sign_message(message, private_key) -> bytes`: Implements RSA-PSS with `hashes.SHA256()` and `padding.MGF1(hashes.SHA256())` with `salt_length = padding.PSS.MAX_LENGTH`.
- `verify_signature(message, signature, public_key) -> bool`: Verifies the signature against the message digest. Catches `InvalidSignature` exceptions internally and returns a clean boolean status (`True`/`False`) to prevent timing side channels or uncontrolled crashes.
- `get_public_key_fingerprint(public_key) -> str`: Computes the canonical SHA-256 hex fingerprint of the public key's DER-encoded `SubjectPublicKeyInfo` representation.

5.2 Genuine Terminal Execution Transcripts

5.2.1 Activity 1: Caesar Cipher Demonstration

```
$ uv run python activity1_caesar/cli.py -p "CNS Lab: Activity-01 @ 2026!" -k 3

================================================================================
           CNS ACTIVITY 1: CAESAR CIPHER DEMONSTRATION
================================================================================
Original Plaintext  : CNS Lab: Activity-01 @ 2026!
Input Key           : 3 (Normalized shift: 3)
Encrypted Ciphertext: FQV Ode: Dfwlylwb-01 @ 2026!
Recovered Plaintext : CNS Lab: Activity-01 @ 2026!
--------------------------------------------------------------------------------
Round-Trip Verified : SUCCESS (Original == Recovered)
================================================================================
```

5.2.2 Activity 2: Vigenère Cipher Demonstration

```
$ uv run python activity2_vigenere/cli.py -p "The quick brown fox jumps over the lazy
    dog!" -k "LEMON"

================================================================================
           CNS ACTIVITY 2: VIGENERE CIPHER DEMONSTRATION
================================================================================
Original Plaintext  : The quick brown fox jumps over the lazy dog!
Input Key           : LEMON (Normalized: LEMON)
Encrypted Ciphertext: Elq ehtgw pezaz tbi ngacd shse elq znkc pct!
Recovered Plaintext : The quick brown fox jumps over the lazy dog!
--------------------------------------------------------------------------------
Round-Trip Verified : SUCCESS (Original == Recovered)
================================================================================
```

5.2.3 Activity 3: Two-Time Pad Vulnerability Demonstration

```
$ uv run python activity3_one_time_pad/cli.py --demo-key-reuse

================================================================================
           DEMONSTRATION: CATASTROPHIC TWO-TIME PAD ATTACK
================================================================================
```

```
Plaintext 1 (M1)        : ATTACK AT DAWN!
Plaintext 2 (M2)        : DEFEND THE WALL
Shared Reused Key (Hex): 10FB4279D7D62311E26CD8393A0648
Ciphertext 1 (C1 Hex)   : 51AF1638949D0350B64C9C786D4869
Ciphertext 2 (C2 Hex)   : 54BE043C99920345AA29F86E7B4A04
--------------------------------------------------------------------------------
Adversary computes C1 ^ C2:
C1 ^ C2 (Hex)           : 051112040D0F00151C65641616026D
M1 ^ M2 (Hex)           : 051112040D0F00151C65641616026D
Key Canceled Out?       : YES (C1 ^ C2 == M1 ^ M2)

Conclusion: Reusing a One-Time Pad key destroys perfect secrecy entirely,
allowing an eavesdropper to recover the plaintext difference without knowing the key!
================================================================================
```

5.2.4 Activity 4: Digital Signature Tamper Detection Demonstration

```
$ uv run python activity4_digital_signature/cli.py --demo-tamper

================================================================================
      DEMONSTRATION: DIGITAL SIGNATURE INTEGRITY & TAMPER DETECTION
================================================================================
Original Message        : Authorise payment of 5000 INR to Student Club.
Tampered Message        : Authorise payment of 50000 INR to Student Club.
--------------------------------------------------------------------------------
1. Authentic Message + Original Signature : VALID [Passed]
2. Tampered Message + Original Signature  : INVALID [Tampering Detected!]
3. Original Message + Corrupted Signature : INVALID [Corruption Detected!]
4. Original Message + Unrelated Public Key: INVALID [Key Mismatch Detected!]
--------------------------------------------------------------------------------
Conclusion: Any modification of the message, signature corruption, or key mismatch
immediately invalidates the digital signature, providing strict integrity assurance.
================================================================================
```

5.3 Character and Byte Data Flow Pipelines

The interactive TUI in `main.py` dynamically renders character-by-character and byte-by-byte data flow pipelines, as documented in Tables 5.1, 5.2, and 5.3.

Input Char	Alphabet Range	Base Pos (P)	Transformation: (P + K) mod 26	Cipher Char (C)
’H’	uppercase [A-Z]	7	$(7+3) \pmod{26} = 10$	’K’
’e’	lowercase [a-z]	4	$(4+3) \pmod{26} = 7$	’h’
’l’	lowercase [a-z]	11	$(11+3) \pmod{26} = 14$	’o’
’l’	lowercase [a-z]	11	$(11+3) \pmod{26} = 14$	’o’
’o’	lowercase [a-z]	14	$(14+3) \pmod{26} = 17$	’r’
’,’	non-alphabetic	N/A	Preserved verbatim	’,’

Table 5.1: Caesar Cipher Character Data Flow Pipeline Trace ($K = 3$)

Plain (P)	Key (K)	Shift	Tabula Recta Operation	Cipher (C)	Stream Pointer
’A’	’L’	+11	$(0+11) \pmod{26} = 11$	’L’	Advanced to Index 1
’T’	’E’	+4	$(19+4) \pmod{26} = 23$	’X’	Advanced to Index 2
’T’	’M’	+12	$(19+12) \pmod{26} = 5$	’F’	Advanced to Index 3
’A’	’O’	+14	$(0+14) \pmod{26} = 14$	’O’	Advanced to Index 4
’ ’	—	0	Preserved unshifted	’ ’	Paused (no advance)

Table 5.2: Vigenère Polyalphabetic Keyword Alignment Pipeline Trace (Key = ’LEMON’)

Byte #	Char	Plain Byte	XOR	CSPRNG Key Byte	Cipher Byte
1	’C’	0x43 (01000011)	$\oplus$	0x9A (10011010)	0xD9 (11011001)
2	’N’	0x4E (01001110)	$\oplus$	0x2B (00101011)	0x65 (01100101)
3	’S’	0x53 (01010011)	$\oplus$	0x7F (01111111)	0x2C (00101100)
4	’ ’	0x20 (00100000)	$\oplus$	0xE1 (11100001)	0xC1 (11000001)
5	’2’	0x32 (00110010)	$\oplus$	0x0C (00001100)	0x3E (00111110)

Table 5.3: One-Time Pad Bytewise XOR Transformation Pipeline Trace

Chapter 6
TESTING METHODOLOGY AND VERIFICATION MATRIX

6.1 Systematic Verification Strategy

Automated verification was conducted using `pytest 9.1.1` configured to auto-discover unit test suites via `pyproject.toml`. Tests span five core categories: (1) **Known-Answer Tests (KAT)** verifying deterministic vectors, (2) **Boundary & Wrapping** validating modular wrap ($Z \to A$), cyclic keys, and null strings, (3) **Formal Invertibility** proving $\mathscr{D}_K(\mathscr{E}_K(P)) = P$, (4) **Negative Type Validation** enforcing robust `TypeError`/`ValueError` handling, and (5) **Tamper Security** proving bit-level corruption triggers verification failure.

6.2 Exhaustive 69-Test Verification Matrix

Table 6.1 documents the empirical outcomes of all 69 automated unit tests.

Table 6.1: Complete 69-Test Verification Matrix Across All Packages

Test Identifier	Category	Input / Condition	Status
Activity 1: Caesar Cipher (`activity1_caesar/tests/test_caesar.py`)			
`test_normalize_key_standard`	Normalization	Key = 3	**PASS**
`test_normalize_key_zero`	Boundary	Key = 0	**PASS**
`test_normalize_key_twenty_five`	Boundary	Key = 25	**PASS**
`test_normalize_key_greater...`	Modulo Wrap	Key = 26, 29 $\to$ 0, 3	**PASS**
`test_normalize_key_negative`	Negative Shift	Key = -1, -3 $\to$ 25, 23	**PASS**
`test_normalize_key_invalid_types`	Input Validation	String, Float, None, Bool	**PASS**
`test_encrypt_known_answers`	KAT	”HELLO” (k=3), ROT13	**PASS**
`test_encrypt_wrap_boundaries`	Wrap Boundary	”XYZ” (k=3) $\to$ ”ABC”	**PASS**
`test_encrypt_key_variations`	Equivalence	Keys 0, 26, 29, -1	**PASS**
`test_encrypt_invalid_plaintext`	Input Validation	Integer 12345, None	**PASS**
`test_encrypt_empty_message`	Edge Case	Empty string ””	**PASS**
`test_encrypt_non_alphabetic`	Symbol Handling	Spaces, digits, punctuation	**PASS**
`test_encrypt_repeated_chars`	Pattern Handling	”AAAA” $\to$ ”BBBB”	**PASS**
`test_decrypt_known_answers`	Decryption KAT	”KHOOR” (k=3) $\to$ ”HELLO”	**PASS**
`test_round_trip_property`	Invertibility	10 pangram variants	**PASS**
`test_decrypt_invalid_types`	Input Validation	Invalid types for decrypt	**PASS**
`test_cli_run_demo`	Integration / CLI	Full workflow execution	**PASS**
Activity 2: Vigenère Cipher (`activity2_vigenere/tests/test_vigenere.py`)			
`test_normalize_key_standard`	Normalization	”LEMON”, ”lemon”, ”LeMoN”	**PASS**
`test_normalize_key_empty`	Input Validation	Empty string ””	**PASS**
`test_normalize_key_non_alpha`	Input Validation	Non-alphabetic tokens	**PASS**
`test_normalize_key_invalid_types`	Type Validation	Integer, None, bool, list	**PASS**
`test_encrypt_known_answers`	KAT	”ATTACKATDAWN” / ”LEMON”	**PASS**
`test_encrypt_single_char_key`	Caesar Equiv.	Key ”D” $\equiv$ Caesar shift 3	**PASS**
`test_encrypt_key_longer...`	Boundary	Plain len 5, Key len 11	**PASS**
`test_encrypt_plaintext_longer...`	Key Repetition	”AAAAAA” / ”ABC”	**PASS**
`test_encrypt_invalid_plaintext`	Type Validation	Plaintext integer, None	**PASS**

Continued on next page...

Table 6.1 – *Continued from previous page*

Test Identifier	Category	Input / Condition	Status
test_encrypt_case_punctuation	Stream Alignment	Key advances only on letters	**PASS**
test_encrypt_no_alphabetic	Edge Case	Digits and symbols only	**PASS**
test_encrypt_empty_plaintext	Edge Case	Empty string ""	**PASS**
test_encrypt_repeated_patterns	Pattern Handling	"AAAA" (k="B"), "ABAB"	**PASS**
test_decrypt_known_answers	Decryption KAT	"LXFOPVEFRNHR" / "LEMON"	**PASS**
test_round_trip_property	Invertibility	8 varied test cases	**PASS**
test_decrypt_invalid_types	Type Validation	Non-string ciphertext	**PASS**
test_cli_run_demo	Integration / CLI	CLI execution test	**PASS**
Activity 3: One-Time Pad (activity3_one_time_pad/tests/test_otp.py)			
test_generate_key_length	CSPRNG Length	0, 10, 100 bytes requested	**PASS**
test_generate_key_independence	Randomness	Two 32-byte successive calls	**PASS**
test_generate_key_invalid_len	Input Validation	Negative length, float, bool	**PASS**
test_xor_bytes_known_vector	Fixed KAT	Deterministic byte vectors	**PASS**
test_xor_bytes_properties	Invertibility	Self-inverse, zero identity	**PASS**
test_xor_bytes_length_mismatch	Length Check	Key length != Data length	**PASS**
test_xor_bytes_invalid_types	Type Validation	String, integer, None	**PASS**
test_encrypt_decrypt_kat	Deterministic KAT	"CNS 2026" / fixed hex key	**PASS**
test_encrypt_accepts_raw_bytes	Interface	Arbitrary binary sequences	**PASS**
test_encrypt_decrypt_round_trip	Fuzz Invertibility	Sentences, symbols, pangrams	**PASS**
test_encrypt_invalid_types	Type Validation	Plaintext int, string key	**PASS**
test_empty_message	Edge Case	0-byte key and message	**PASS**
test_multibyte_utf8_chars	Unicode / UTF-8	Accents, kanji, lock emoji	**PASS**
test_decrypt_incorrect_key	Security Property	Wrong key yields raw bytes	**PASS**
test_two_time_pad_attack	Cryptanalysis	$C_1 \oplus C_2 == M_1 \oplus M_2$	**PASS**
test_cli_run_demo	Integration / CLI	Standard CLI run	**PASS**
test_cli_demo_key_reuse	Integration / CLI	Two-Time Pad CLI demo	**PASS**
Activity 4: Digital Signature (activity4_digital_signature/tests/test_signature.py)			
test_generate_key_defaults	Key Generation	2048-bit RSA, $e = 65537$	**PASS**
test_generate_key_custom_size	Custom Size	1024-bit generation	**PASS**
test_generate_key_invalid_size	Validation	< 1024 bits, string, None	**PASS**
test_export_public_key_pem	Serialization	SubjectPublicKeyInfo PEM	**PASS**
test_get_public_key_fingerprint	Identification	64-character SHA-256 hex	**PASS**
test_sign_and_verify_normal	Functional	Standard string message	**PASS**
test_sign_accepts_raw_bytes	Interface	Binary byte message	**PASS**
test_sign_invalid_types	Type Validation	Integer message, public key	**PASS**
test_verification_message_tamper	Tamper Detection	Modified message content	**PASS**
test_verification_sig_tamper	Tamper Detection	Corrupted signature bit	**PASS**
test_verification_wrong_pubkey	Key Mismatch	Independent public key	**PASS**
test_verification_malformed_sig	Robustness	Truncated/non-bytes sig	**PASS**
test_probabilistic_signature_pss	PSS Randomness	Same msg $\rightarrow$ distinct sigs	**PASS**
test_empty_message_signing	Edge Case	Empty string ""	**PASS**
test_multibyte_unicode_signing	Unicode / UTF-8	Accents, kanji, emoji	**PASS**
test_verify_invalid_pubkey	Validation	Non-RSAPublicKey type	**PASS**
test_cli_run_demo	Integration / CLI	Standard CLI workflow	**PASS**
test_cli_demo_tamper	Integration / CLI	4-way tamper CLI demo	**PASS**

Summary of Test Execution: Discovered: 69; Passed: 69 (100%); Failed: 0; Duration: 0.55s.

Chapter 7
SECURITY ANALYSIS AND CRYPTANALYSIS

7.1 Monoalphabetic Cryptanalysis

The Caesar cipher provides zero computational confidentiality against modern adversaries due to two fatal mathematical weaknesses:

1. **Exhaustive Key Search (Brute-Force Attack)**: The keyspace is strictly bounded by $|\mathscr{K}| = 26$. Because $K \equiv 0 \pmod{26}$ is the identity map, only 25 non-trivial keys exist. A modern CPU evaluates all 25 candidate plaintexts in less than 5 microseconds:

$$T_{\text{break}} = \mathscr{O}(|\mathscr{K}| \cdot N) = 25 \cdot N \text{ operations} \tag{7.1}$$

2. **Frequency Analysis**: The cipher preserves natural language letter frequencies. In natural English text, the letter **'E'** occurs with $\approx 12.7\%$ frequency, followed by **'T'** (9.1%), **'A'** (8.2%), and **'O'** (7.5%). An attacker computes the relative unigram frequency distribution of the ciphertext and aligns the maximum peak with 'E', immediately discovering the shift key K without testing remaining keys.

7.2 Polyalphabetic Cryptanalysis

Although the Vigenère cipher expands the keyspace to 26^L for a keyword of length L, it remains fundamentally broken against classical cryptanalysis:

1. **Kasiski Examination**: Because the keyword repeats periodically every L characters, repeated words in the plaintext separated by a distance that is a multiple of L will be enciphered with the exact same key sequence, producing identical ciphertext segments. By measuring the distances $d_1, d_2, \ldots, d_m$ between identical n-grams (trigrams or four-grams) and computing their greatest common divisor:

$$L = \gcd(d_1, d_2, \ldots, d_m) \tag{7.2}$$

   the attacker deduces the keyword length L.

2. **Friedman's Index of Coincidence (I_c)**: The Index of Coincidence measures the probability that two randomly selected letters from a text are identical:

$$I_c = \frac{\sum_{i=0}^{25} f_i(f_i - 1)}{N(N-1)} \tag{7.3}$$

   For standard English text, $I_{c,\text{English}} \approx 0.067$, while for a purely random uniform distribution, $I_{c,\text{Random}} \approx 1/26 \approx 0.0385$. Using William Friedman's formula, the keyword

length L is estimated directly from the ciphertext:

$$L \approx \frac{0.027 \cdot N}{(N-1)I_c - 0.038N + 0.065} \tag{7.4}$$

3. **Caesar Decomposition**: Once L is established, the ciphertext is partitioned into L independent slices:

$$\mathscr{S}_j = \{C_j, C_{j+L}, C_{j+2L}, \dots\} \quad \text{for } j \in \{0, 1, \dots, L-1\} \tag{7.5}$$

Each slice $\mathscr{S}_j$ is an isolated monoalphabetic Caesar cipher! The adversary solves all L slices in parallel via standard unigram frequency analysis.

7.3 Information-Theoretic Perfect Secrecy

7.3.1 The Five Mandatory Conditions for Shannon's Perfect Secrecy

The mathematical guarantee $\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m)$ holds if and only if all five conditions are strictly observed:

1. **Uniform Randomness**: Each key byte must be generated from a true random source or high-entropy CSPRNG with zero statistical bias: $\mathbb{P}(K_i = k) = 1/256$.
2. **Key Length Equality**: The key length must strictly satisfy $|K| \geq |M|$.
3. **Absolute Confidentiality**: The key material must remain completely unknown to unauthorized third parties.
4. **Strict Single Use**: Key material must be used **strictly once** and immediately zeroized.
5. **Secure Pre-Shared Distribution**: Communicating parties must securely synchronize key pads prior to message transmission.

7.3.2 Catastrophic Two-Time Pad Vulnerability

If an identical key K is reused to encrypt two separate plaintexts M_1 and M_2:

$$C_1 = M_1 \oplus K, \quad C_2 = M_2 \oplus K \tag{7.6}$$

An eavesdropper intercepting both ciphertexts computes their bitwise XOR sum:

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2 \oplus (K \oplus K) = M_1 \oplus M_2 \tag{7.7}$$

The key cancels out entirely! The adversary now possesses the XOR sum of two natural language plaintexts. Because ASCII spaces (`0x20`) XORed with alphabetic characters flip their case, the adversary applies **crib dragging** (sliding common English words such as `" the "`, `" that "`, `" with "` across $C_1 \oplus C_2$) to simultaneously recover both plaintexts without ever determining the key.

7.4 Asymmetric Signature Security and PKI

7.4.1 Security Properties Provided

- **Authenticity**: Verification confirms the message was signed by the specific private key corresponding to the public key.
- **Integrity**: Modifying even a single bit of message M produces an entirely different SHA-256 digest h', causing PSS verification to fail immediately.
- **Non-Repudiation**: Because private exponent d is known only to the signer, they cannot credibly deny authorship of a valid signature.

7.4.2 RSA-PSS vs. Legacy PKCS#1 v1.5 Padding

In legacy PKCS#1 v1.5, deterministic formatting allows Bleichenbacher-style chosen-ciphertext attacks and signature malleability. In contrast, **RSA-PSS (RFC 8017)** incorporates a randomized salt, making the signature probabilistic: signing the exact same message twice generates distinct 256-byte signature blocks, both of which verify correctly. RSA-PSS possesses a tight security reduction to the RSA problem in the Random Oracle Model.

7.4.3 Comparative Evaluation Matrix

Table 7.1 synthesizes the comparative properties across all four evaluated cryptosystems.

Algorithm	Core Strengths	Theoretical Limitations	Engineering Constraints
Caesar	$\mathcal{O}(N)$ complexity; trivial mental computation; foundational education.	Tiny keyspace ($\lvert\mathcal{K}\rvert = 25$); preserves letter frequencies.	Alphabet-only; zero security for modern communication.
Vigenère	Flattens unigram frequencies; keyspace scales as 26^L.	Periodic key leaks length L via Kasiski/Friedman tests.	Requires alphabetic stream synchronization; easily broken.
One-Time Pad	Information-theoretic perfect secrecy ($H(M\lvert C) = H(M)$).	Key length must equal message length; key reuse catastrophic.	Severe key distribution and storage bottleneck; pad zeroization.
RSA-PSS	Asymmetric public verification; non-repudiation; message integrity.	Vulnerable to Shor's algorithm on quantum computers.	High modular exponentiation overhead; requires PKI for identity.

Table 7.1: Comparative Evaluation of Implemented Cryptosystems

Chapter 8
APPLICATIONS AND CONCLUSION

8.1 Historical and Modern Applications

Historical Applications:

- **Julius Caesar (58 BC)**: Deployed monoalphabetic shifts ($K = 3$) to protect Roman dispatches against illiterate adversaries.
- **American Civil War (1861–1865)**: Confederate forces used Vigenère brass disks for battlefield orders; intercepted and broken by Union analysts.
- **Washington–Moscow Hotline (1963)**: Established post-Cuban Crisis, using teleprinter One-Time Pads with pouch-delivered tapes.

Modern Industry Applications:

- **ROT13 Text Obfuscation**: A Caesar shift ($K = 13$) obscures spoilers and puzzle solutions in online forums and software registries.
- **TLS 1.3 Transport Layer Security**: Digital signatures (RSA-PSS, Ed25519) authenticate web servers during handshakes, mitigating MitM attacks.
- **Software Code Signing**: Microsoft Authenticode and Apple Gatekeeper utilize RSA-PSS and SHA-256 to verify binary authenticity prior to OS execution.
- **Quantum Key Distribution (QKD)**: BB84 protocols combine photon polarization key exchange with One-Time Pad stream encryption.

8.2 Conclusion

This report presented an empirical implementation, verification, and cryptanalytic audit of four cryptographic primitives for Batch `2026_CNS_A_01` at K. S. Institute of Technology. The software audit establishes:

1. Classical substitution ciphers (Caesar, Vigenère) demonstrate modular residue rings $\mathbb{Z}_{26}$ but fail completely against automated frequency and periodicity cryptanalysis.
2. The One-Time Pad mathematically guarantees unconditional information-theoretic secrecy, but utility is constrained by key distribution, synchronization, and storage overhead.
3. Asymmetric signatures (RSA-PSS with SHA-256) deliver robust message integrity and non-repudiation across untrusted networks when coupled with Public Key Infrastructure (PKI).
4. All 69 automated unit tests passed with 100% correctness in 0.55 seconds, confirming full compliance with Course Outcomes CO1, CO3, and CO4.

REFERENCES

1. Shannon, C. E. (1949). *"Communication Theory of Secrecy Systems"*. Bell System Technical Journal, 28(4), 656–715.
2. Diffie, W., & Hellman, M. (1976). *"New Directions in Cryptography"*. IEEE Transactions on Information Theory, 22(6), 644–654.
3. Rivest, R. L., Shamir, A., & Adleman, L. (1978). *"A Method for Obtaining Digital Signatures and Public-Key Cryptosystems"*. Communications of the ACM, 21(2), 120–126.
4. Moriarty, K., Kaliski, B., Jonsson, J., & Rusch, A. (2016). *"PKCS #1: RSA Cryptography Specifications Version 2.2"* (RFC 8017). Internet Engineering Task Force (IETF).
5. National Institute of Standards and Technology (NIST). (2015). *"Secure Hash Standard (SHS)"* (FIPS PUB 180-4). U.S. Department of Commerce.
6. Bellare, M., & Rogaway, P. (1996). *"The Exact Security of Digital Signatures: How to Sign with RSA and Rabin"*. Advances in Cryptology – EUROCRYPT '96, LNCS 1070, 399–416.
7. Stallings, W. (2017). *"Cryptography and Network Security: Principles and Practice"* (7th ed.). Pearson Education.
8. Ferguson, N., Schneier, B., & Kohno, T. (2010). *"Cryptography Engineering: Design Principles and Practical Applications"*. John Wiley & Sons.
9. Python Software Foundation. (2026). *"secrets – Generate cryptographically strong random numbers"*. Python Standard Library Documentation.
10. Python Cryptographic Authority (PyCA). (2026). *"cryptography: Asymmetric Algorithms and Hazard Materials"*. Available at: https://cryptography.io/.
11. Kasiski, F. W. (1863). *"Die Geheimschriften und die Dechiffrir-Kunst"*. E. S. Mittler und Sohn, Berlin.
12. Friedman, W. F. (1922). *"The Index of Coincidence and Its Applications in Cryptanalysis"*. Riverbank Laboratories, Publication No. 22.

GROUP CONTRIBUTION MATRIX

In accordance with course guidelines for Batch 2026_CNS_A_01, the distribution of responsibilities across the group members is recorded in Table 8.1.

Sl.	USN	Student Name	Primary Technical Responsibilities	Verified Deliverables
1	1KS23CS059	**Jayaditya Dev**	Interactive TUI architecture (main.py), Activity 4 RSA-PSS implementation, MGF1 padding, SHA-256 fingerprinting, PKI analysis.	main.py, activity4_digital_signature/, 18 tests.
2	1KS23CS043	**Durgashree M**	Activity 3 One-Time Pad architecture, CSPRNG randomness verification (secrets), Shannon perfect secrecy proof, Two-Time Pad attack module.	activity3_one_time_pad/, Two-time pad demo, 17 tests.
3	1KS23CS020	**Aryaman Tiwari**	Activity 2 Vigenère cipher architecture, Tabula Recta mapping, alphabetic stream synchronization, Kasiski and Friedman cryptanalysis.	activity2_vigenere/, CLI demonstration suite, 17 tests.
4	1KS23CS034	**Chirag T**	Activity 1 Caesar cipher architecture, modulo 26 normalization, boundary wrapping, exhaustive key search analysis, frequency analysis model.	activity1_caesar/, CLI demonstration suite, 17 tests.

Table 8.1: Group Contribution and Deliverable Ownership Matrix